Reading Between the Lines: Assessing LLM Comprehension of Scientific Code

Eliza Berman[1] **Shreya Jha**[1] **Lohit Jagarapu**[1] **Rohan Subramanian**[1]
[1] Courant Institute, New York University

Abstract

Large language models (LLMs) are increasingly used to write, review, and debug scientific simulation code, but it remains unclear what evidence they rely on when interpreting code that represents a physical system. We study this question using Python solvers for partial differential equations (PDEs). We introduce a controlled dataset of PDE solver snippets spanning Heat, Wave, Burgers, and Navier–Stokes equations. We apply systematic perturbations that remove or corrupt comments, obfuscate variable names, and introduce physically invalid behavior while preserving syntactic validity. We evaluate models through free-generation accuracy, multiple-choice confidence, hidden-state probes, and belief revision after execution summaries. Model outputs are highly sensitive to lexical cues: variable-name and comment perturbations reduce accuracy and increase uncertainty, especially for PDE-class and validity judgments. Yet hidden-state probes show that PDE class remains decodable even when lexical baselines weaken, suggesting that models encode physical information not always reflected in their stated answers. Execution summaries only partially correct these judgments, with revisions concentrated in variable-obfuscation conditions where lexical priors are weakest. Overall, LLMs encode some physical structure in scientific code, but their expressed judgments remain strongly shaped by names, comments, and other surface cues.

1 Introduction

Large Language Models (LLMs) are increasingly used as coding assistants, with rapid recent progress in code explanation, debugging, and generation. [Weisz et al., 2025]. Despite these advances, it remains unclear how LLMs comprehend code and which sources of meaning they rely on when making judgments about a program. Prior work on code understanding has used large-scale benchmarks containing primarily generic software-engineering or competitive programming tasks [Du et al., 2023, Jain et al., 2024], and focused on behavioral evaluations of static code. However, scientific simulation code raises a different kind of challenge: correct interpretation often requires understanding not only what the code does syntactically but also what physical system it is intended to simulate.

We study this problem in the context of partial differential equation (PDE) solvers. PDEs are central to modeling physical systems such as diffusion, waves, and fluid dynamics, but implementing them requires domain-specific knowledge about equations, numerical methods, stability, and physical validity. PDE solver code is especially useful for evaluating LLM code comprehension because small changes to signs, terms, timesteps, or boundary conditions can leave code syntactically valid while changing the simulated system or producing nonphysical behavior. Its meaning is also conveyed through multiple sources of evidence: lexical cues such as names and comments, structural cues such as update equations and numerical methods, and execution evidence from the simulated trajectory. This motivates our central question: *When an LLM is given a PDE simulation code, do its outputs and internal representations reflect meaningful physical concepts and structures, or are they primarily determined by coding conventions and syntactic patterns?*

Preprint.

We introduce a controlled dataset of PDE simulation snippets spanning Heat, Wave, Burgers, and Navier–Stokes equations, with perturbations that corrupt comments, obfuscate identifiers, or break physical validity while preserving syntactic validity. We evaluate ten open-weight LLMs in three ways: behavioral prompts measure PDE class, numerical method, physical behavior, and validity; linear probes test whether these attributes are encoded in hidden states; and a belief-revision experiment tests whether a frontier model updates after receiving execution summaries.

Together, our results suggest a gap between what models encode and what they use when answering. Lexical perturbations substantially degrade stated outputs, especially for PDE class and physical validity. However, PDE class remains decodable from hidden states, suggesting that models retain some physical information even when their outputs become less reliable. Execution summaries only partially close this gap: models sometimes revise incorrect judgments, but often remain anchored to their initial code-based interpretation.

Contributions. (1) We introduce a controlled PDE-code dataset designed to study lexical cues, structural code cues, and executable scientific behavior in LLM scientific-code comprehension across four equation classes, three numerical-method families, and four physical behaviors. (2) We evaluate ten open-weight LLMs across behavioral outputs, hidden representations, and belief revision with execution evidence, allowing us to compare what models say, what they encode, and how they update their beliefs. (3) We show that models rely heavily on lexical cues in their stated judgments, while still encoding some physical structure internally. Reasoning models are more robust, but show the same failure modes. Finally, execution evidence partially overcomes code-based priors.

2 Related Work

Modern LLMs perform strongly on many code-generation and code-understanding tasks, but scientific simulation code remains challenging because correctness depends on domain knowledge, numerical stability, and physical validity. Recent work has explored using LLMs to generate or assist with scientific solvers, including PDE solver code [Li et al., 2026] and physics-informed neural networks [He et al., 2025]. These works demonstrate LLMs' capabilities in scientific programming, but focus primarily on generation. In contrast, our work studies comprehension: when a model is given existing PDE solver code, what information does it rely on to identify and judge the simulated system?

Several recent papers investigate how LLMs derive meaning from code. Le et al. [2025] argue that programs convey meaning through both structural semantics (syntax, control flow, execution behavior), which determine program behavior, and human-interpretable naming (identifier names, comments), which communicates intent. They show that semantics-preserving obfuscations, such as removing meaningful names, reduce LLM performance on code summarization and execution prediction tasks. Other work has used identifier obfuscation to improve pretraining efficiency for LLMs [Lachaux et al., 2021, Paul et al., 2025], but it remains unclear how sensitive most modern open-weight or frontier models are to perturbations on scientific code.

A related line of work studies whether LLM representations encode physical information. Song et al. [2025] analyze model residual-stream activations when LLMs are given trajectory data from dynamical systems, finding features correlated with physical quantities such as energy. We ask a complementary question: whether physically meaningful information is encoded when the input is not trajectory data alone, but PDE solver code. This distinction matters because code contains both executable structure and surface-level cues such as comments and variable names.

Finally, recent work has investigated how LLMs revise beliefs while considering different sources of information such as text-based priors and dynamic feedback [Kumaran et al., 2026]. This perspective is missing from prior work on how LLMs derive meaning from code, and more accurately reflects the feedback and evidence-driven environments in which agentic coding LLMs operate.

Together, these works show that LLMs are sensitive to naming in generic code and can encode physical information from trajectory data. However, it remains unclear which cues LLMs rely on when interpreting scientific simulation code itself. We address this gap with a controlled PDE-code dataset that perturbs comments, variable/function names, and physical validity, allowing us to compare model behavior, internal representations, and belief revision under execution evidence.

3 Methods

3.1 Dataset Construction

We curate a dataset of Python PDE solvers using standard scientific-computing libraries such as `numpy`, `scipy`, and `JAX`. Solvers were pulled from pre-written GitHub repositories (references in Appendix A.1) and manually verified by two authors. The dataset contains solvers for four PDE classes: Burgers, Heat, Navier–Stokes, and Wave equations. As summarized in Table 1, the snippets vary substantially in length, numerical method, and dominant physical behavior. Starting from 16 total base Python PDE solvers, we generate controlled variants that alter different sources of evidence available to the model. This design allows us to compare model behavior when surface-level cues are removed or misleading, while also testing whether models can detect physically invalid implementations.

PDE	Snippets	Avg Lines	Avg Chars	Methods Covered	Behaviors Covered
Burgers	4	90 (62–134)	970 (626–1,491)	Explicit	Advection, Diffusion
Heat	4	70 (62–74)	585 (501–686)	Explicit, Implicit	Diffusion
Navier-Stokes	4	243 (174–340)	4,775 (3,204–6,398)	Explicit, Implicit, Spectral	Advection, Diffusion, Restoration
Wave	4	123 (78–175)	3,076 (856–7,462)	Explicit, Spectral	Oscillation, Restoration
Total	**16**	131	2,351		

Table 1: Ground truth commented dataset statistics by PDE type. Each base snippet is replicated across 8 conditions, yielding 32 rows per PDE type (128 total).

We consider four perturbation operations:

1. **Comment removal:** all comments are removed from the solver, eliminating natural-language explanations while leaving executable code unchanged.
2. **Comment corruption:** comments are replaced with comments drawn from a solver for a different PDE class and numerical method. This creates a conflict between the code and its natural-language description; details are provided in Appendix A.1.
3. **Variable/function obfuscation:** variable names are replaced with uninformative identifiers (`foobar1`, `foobar2`, etc.), and function names are replaced with identifiers such as `function1`. The renaming is consistent within each snippet, so logic is conserved.
4. **Physical invalidity:** the snippet is modified to produce physically invalid behavior (i.e. nonphysical trajectories, numerical blow-up, or NaNs) during simulation, while remaining syntactically valid (no execution errors).

Perturbation	Comments	Variables	Valid
Clean+Comment	✓	✓	✓
Clean, No Comment	○	✓	✓
Corrupt Comment	✗	✓	✓
Corrupt Variable	○	✗	✓
Invalid+Comment	✓	✓	✗
Invalid, No Comment	○	✓	✗
CorrComment+Invalid	✗	✓	✗
CorrVar+Invalid	○	✗	✗

Table 2: Dataset perturbations. In the Comments column, ✓ denotes informative comments, ✗ denotes corrupted comments, and ○denotes no comments. In the Variables and Valid columns, ✓/✗ denote ground-truth vs. corrupted variables and valid vs. invalid code, respectively.

We combine these perturbations into eight modification types, shown in Table 2. Each base solver appears once in each modification type, producing 128 total snippets: 16 base solvers $\times$ 8 variants.

3.2 Experimental Framework

Behavioral Evaluation. We first evaluate what models explicitly report when given a PDE solver. We prompt ten LLMs with each of the 128 code snippets and ask for four fields: PDE class, numerical method, dominant physical process, and physical validity. The model suite spans general-purpose, coding-specialized, and reasoning-specialized models from 7B to 70B parameters (models listed in Table 4). All models use `temperature=0.0`; full prompts are provided in Appendix A.3.1. We do not limit generation length; `max_tokens=2048` for non-reasoning models and 16384 for reasoning models. We score PDE class and validity by keyword/alias accuracy. Since numerical method and

physical behavior may have multiple correct labels (up to three), we score these fields by partial recall over the ground-truth label set. We also classify validity responses by hedge-phrase presence, separating certain and uncertain yes/no judgments. Full parsing rules and aliases are in Appendix A.4.

To obtain confidence estimates under fixed answer choices, we also run a multiple-choice version of the evaluation. PDE class is evaluated as a four-way question, while method, behavior, and validity are evaluated as True/False questions over candidate labels. We shuffle answer choices to reduce positional bias and extract first-token log probabilities (`max_tokens=1`) to compute $\log P(\text{correct})$ and entropy. Full prompts and details are provided in Appendix A.3.2. We maintain `temperature=0.0`.

We also analyze the amount of contextual information needed before the model begins to predict the corrupted variable name with high likelihood over the ground truth canonical variable name. A detailed experimental design and results are shown in Appendix A.5.

Representation probing. We test whether physically meaningful attributes of PDE code are encoded in model hidden states. For each snippet, we run a forward pass without generation using the same code-analysis prompt as in Appendix A.3.1, and extract hidden states at code-token positions for each layer. We convert these hidden states into two representations: a mean-pooled representation across code tokens and a final-code-token representation.

For each layer and pooling method, we train logistic regression probes to predict PDE class and physical validity. We use `Qwen2.5-Coder-7B`, `Qwen2.5-Coder-32B`, and `QwQ-32B`, focusing on `Qwen2.5-Coder-7B` in the main text. We use L2 regularization since the input vectors are high-dimensional relative to the dataset size. Full hyperparameter details are provided in Appendix A.6. To prevent probes from exploiting near-duplicate variants of the same base solver and given the size of the dataset, we use leave-one-group-out cross-validation grouped by base snippet. Each fold holds out all eight variants of one base solver and trains on the remaining 120 examples. We report accuracy for PDE class and AUROC for binary labels, with 95% confidence intervals. We compare these probes to a TF-IDF bag-of-words baseline trained and evaluated with the same split and strategy.

Belief revision with execution summaries. We test whether a frontier model revises its initial code-based judgment when shown evidence from executing the solver. We use a two-stage prompting setup with `Gemini-2.5-Flash` [Google DeepMind, 2025], a frontier model accessed through the Google GenAI API. In *Stage 1*, the model receives a formatted PDE code snippet and predicts the same four fields as in the behavioral evaluation: PDE class, numerical method, physical process, and validity. In *Stage 2*, the same conversation continues, and the model receives a structured execution summary and is asked to re-evaluate its answers (full prompt in Appendix A.3.3). We compute accuracy at both stages (as done in *Behavioral Evaluation*) and categorize answer transitions to measure belief revision. Both stages use `temperature=0.0` and `thinking_budget=0.0`.

Execution summaries are precomputed by running each script in a sandboxed environment and extracting numerical diagnostics such as whether the script completes, whether the solution contains NaNs or infinities, and how the maximum absolute value changes over time. We use precomputed summaries rather than live tool use to ensure that every model judgment is evaluated against the same execution evidence. More execution-summary details are provided in Appendix A.7.

4 Discussion

Behavior: LLM outputs rely heavily on lexical cues. Our behavioral experiments ask which cues LLMs rely on when answering physically meaningful questions about PDE code. Figure 1 shows that performance is highest on clean, valid code and degrades when comments or variable names are removed or corrupted. The strongest drop occurs under variable/function obfuscation: PDE-type average LLM accuracy falls from 86.2% on clean commented code to 44.4% under corrupted variables with no comments, and validity accuracy falls from 77.5% to 57.9%. Because this perturbation preserves the program's logic while removing human-interpretable names, this suggests that models use variable and function names as important evidence when interpreting scientific code.

Models often classify physically invalid snippets as valid: even under invalid-code conditions, the majority of validity judgments remain "yes" or uncertain "yes" in several settings. The certainty breakdown (as shown in Figure 1) shows that corrupted comments reduce confident-valid responses on invalid code by 15.6 pp, but this uncertainty is often not strong enough to change the final answer. These results suggest that models have strong code-based priors about how plausible PDE solver code should look.

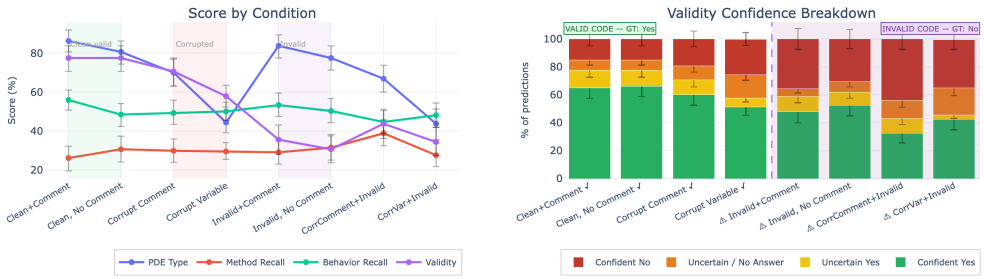

Figure 1: Model performance and validity-confidence across perturbations. Left: mean task scores across LLMs for PDE type, method, behavior, and validity. Performance drops most under variable/function obfuscation and corrupted comments. Right: validity judgments split by confidence using hedge-phrase detection. Models often predict validity for physically invalid code, though uncertainty increases under lexical perturbations. Error bars show 95% bootstrap confidence intervals.

Reasoning models achieve higher average scores than non-reasoning models across perturbation types (Figure 2). The reasoning-model advantage is largest in the corrupted-comment, corrupted-variable, and invalid-code settings, with an average gap of 11–22 pp depending on condition. The same perturbations still reduce performance.

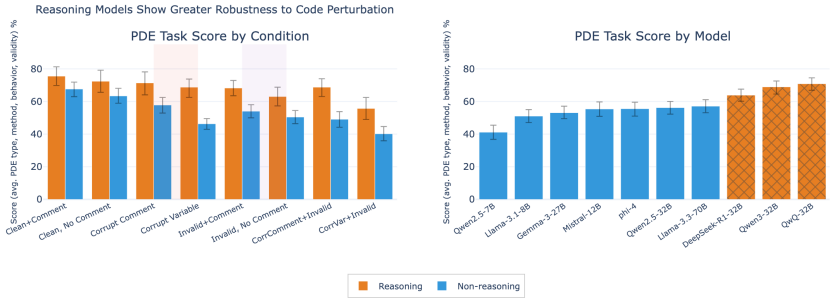

Figure 2: Average score across all free-generation tasks versus perturbation type, grouped by model type. Error bars show 95% bootstrap confidence intervals. Reasoning models outperform non-reasoning models across all perturbations.

The multiple-choice confidence results (Figure 3) support the same conclusion that perturbations destabilize model judgments. Corrupted variables and corrupted comments produce the largest decreases in $\log P(\text{correct})$ and the largest entropy increases, especially for PDE-class classification. This shows that when familiar names and comments are removed or made misleading, models become less confident in the correct scientific interpretation.

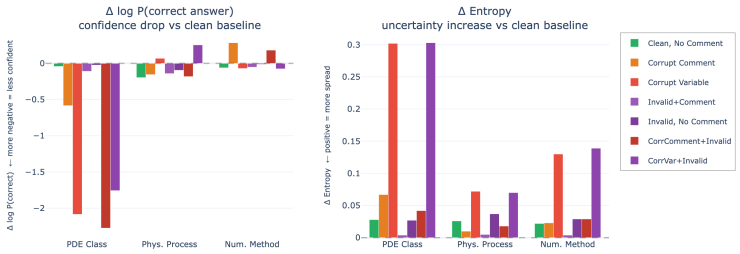

Figure 3: Multiple-choice confidence shifts under code perturbations. Bars show mean change in $\log P(\text{correct})$ (left) and Shannon entropy (right) relative to the clean, commented baseline. Negative $\Delta \log P(\text{correct})$ indicates reduced confidence in the correct answer; positive Δ entropy indicates greater uncertainty. Corrupted variables and comments produce the largest confidence drops and uncertainty increases, especially for PDE-class classification.

Interpretability: models encode physical information that they do not always use. The linear-probe experiment asks whether physically meaningful attributes are encoded in the model's residual stream, and whether these encodings depend on lexical cues. Hidden states still contain decodable PDE information even when lexical cues are weakened (Figure 4). PDE class remains linearly decodable from `Qwen2.5-Coder-7B` hidden states at near 80% accuracy at most perturbation conditions, including settings where the bag-of-words baseline weakens. This suggests the model's internal representations contain PDE class information. However, this encoded information does not fully predict behavior, as behavioral performance still degrades under lexical perturbation. Physical validity shows a weaker pattern than PDE class: probe performance is closer to 60% accuracy. However, AUROC is higher and displays a clearer trend, indicating the probe can rank validity probabilities despite a miscalibrated threshold for classification. Probe performance drops for perturbations to comments and variables. This weak encoding of validity is consistent with the behavioral finding that models shift towards uncertain or invalid predictions for the invalid-code perturbations, but not strongly enough to change the final answer.

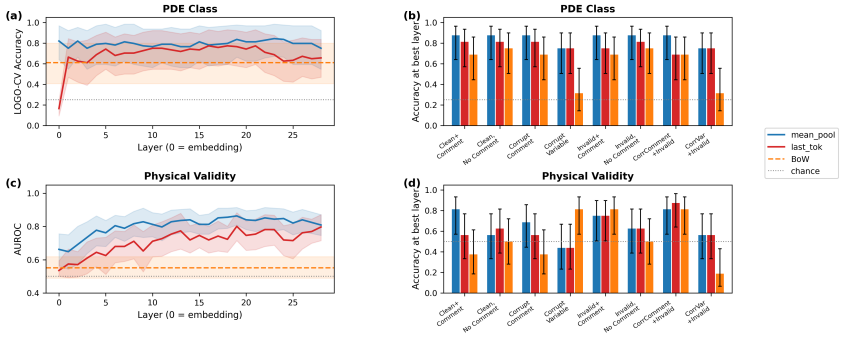

Figure 4: Linear probe performance on `Qwen2.5-Coder-7B` for PDE class (top) and physical validity (bottom), comparing mean-pooled (blue) and last-token (red) representations against a bag-of-words baseline and chance, shown across layers (left) and at the best layer per dataset condition (right).

Code execution: models only partially revise code-based priors. We ask whether a frontier model can revise its initial code-based judgment after seeing execution evidence. `Gemini-2.5-Flash` never transitions from a correct to an incorrect answer in this dataset after receiving execution summaries, but occasionally corrects incorrect answers (Figure 5), improving its overall accuracy by 7.8 pp after execution evidence. The largest gains occur in the variable-obfuscation settings (12.5 pp for valid cases and 18.8 pp for invalid cases). We posit that when meaningful names are removed, the model's code prior is weaker, so execution evidence receives more weight. Alternatively, the execution summary may restore some semantic information lost through obfuscation by identifying the main solution and describing its behavior. Importantly, execution evidence does not correct all wrong judgments: 40% of all examples remain incorrect. Gemini also differs from our open-weight model suite in its validity bias, appearing more willing to classify code as invalid even in valid conditions, suggesting that different model families may attend to different aspects of the same code under identical prompting. These results also suggest that invalidity identification differs by type: failures with NaNs or infinities are easier to identify than subtler finite-valued numerical errors, where trajectories may behave inconsistently without obvious nonfinite values (see Appendix A.7.2).

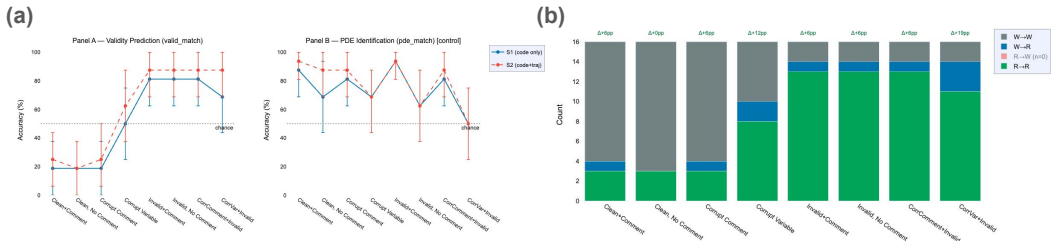

Figure 5: `Gemini-2.5-Flash` belief revision across perturbation types. (a) Validity-prediction and PDE-identification accuracy under code-only (S1) and code+summary (S2) prompting. (b) Counts of belief-revision transitions for validity (Right→Right, Wrong→Right, Right→Wrong, Wrong→Wrong) with accuracy change annotated above each bar.

Limitations. This study is a pilot evaluation of LLM comprehension of PDE code. The dataset is small, with 16 base snippets and 128 variants, and some confounds remain across PDE class, code style, external library usage, and snippet length. These factors limit fine-grained conclusions about individual numerical methods or physical behaviors, so we focus on broad trends with confidence intervals rather than isolated condition-level differences. Probe results are especially difficult to interpret given the small sample size relative to dimensionality, and are reported as preliminary results with appropriate confidence intervals. The code-execution experiment uses a single frontier model that displays opposite validity bias compared to open-weight models evaluated, leaving broader frontier-model comparisons to future work. Further work with frontier and reasoning models could also provide insight into why validity is a difficult target that models have different biases about.

5 Conclusion and Future Work

We introduce a controlled PDE-code dataset with perturbations to comments, variable/function names, and physical validity, and use it to evaluate LLM scientific-code comprehension across behavioral outputs, hidden-state probes, and belief revision with execution summaries. Our results suggest that models rely heavily on lexical cues in their stated judgments, while still encoding some physical information internally, particularly PDE class. Physical validity is harder: it is less robustly encoded and more sensitive to surface cues. Our work provides useful insights for researchers who regularly use LLMs for writing scientific simulation code. Future work should expand the dataset across more PDE classes, solver styles, and invalidity types, evaluate additional frontier models, and study agentic debugging settings in which models choose what to run or inspect rather than receiving fixed execution summaries.

References

[1] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation, 2023. URL https://arxiv.org/abs/2308.01861.

[2] Google DeepMind. Gemini 2.5 flash model documentation, 2025. URL https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash.

[3] X. He, L. You, H. Tian, B. Han, I. Tsang, and Y.-S. Ong. Lang-pinn: From language to physics-informed neural networks via a multi-agent framework. *arXiv preprint arXiv:2510.05158*, 2025.

[4] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

[5] D. Kumaran, S. M. Fleming, L. Markeeva, J. Heyward, A. Banino, M. Mathur, R. Pascanu, S. Osindero, B. De Martino, P. Veličković, and V. Patraucean. Competing biases underlie overconfidence and underconfidence in llms. *Nature Machine Intelligence*, 8(4):614–627, Apr. 2026. ISSN 2522-5839. doi: 10.1038/s42256-026-01217-9. URL http://dx.doi.org/10.1038/s42256-026-01217-9.

[6] M.-A. Lachaux, B. Roziere, M. Szafraniec, and G. Lample. Dobf: A deobfuscation pre-training objective for programming languages. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 14967–14979. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/7d6548bdc0082aacc950ed35e91fcccb-Paper.pdf.

[7] C. C. Le, M. V. Pham, C. D. Van, H. N. Phan, H. N. Phan, and T. N. Nguyen. When names disappear: Revealing what llms actually understand about code. *arXiv preprint arXiv:2510.03178*, 2025.

[8] S. Li, T. Marwah, J. Shen, W. Sun, A. Risteski, Y. Yang, and A. Talwalkar. Codepde: An inference framework for llm-driven pde solver generation. *Transactions of Machine Learning Research*, 2026. doi: 10.48550/ARXIV.2505.08783. URL https://arxiv.org/abs/2505.08783.

[9] I. Paul, H. Yang, G. Glavaš, K. Kersting, and I. Gurevych. Obscuracoder: Powering efficient code lm pre-training via obfuscation grounding. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 13608–13646, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/23ff02034404b65776080cbf7148addd-Paper-Conference.pdf.

[10] Y. Song, J. Bae, D.-K. Kim, and H. Jeong. Uncovering emergent physics representations learned in-context by large language models. *arXiv preprint arXiv:2508.12448*, 2025.

[11] J. D. Weisz, S. V. Kumar, M. Muller, K.-E. Browne, A. Goldberg, K. E. Heintze, and S. Bajpai. Examining the use and impact of an ai code assistant on developer productivity and experience in the enterprise. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, CHI EA '25, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400713958. doi: 10.1145/3706599.3706670. URL https://doi.org/10.1145/3706599.3706670.

A Appendix

A.1 Dataset generation

Sources of PDE solvers.

PDE	Sources
Heat Equation	Gilbert Francois - Heat Equations
Wave Equation	Gilbert Francois - Wave Equations;
	Sacha Binder - Wave Equation Simulations
Burgers Equation	Gilbert Francois - Burgers Equations
Navier Stokes Equation	PDEBench - Compressible Fluids
	JAX-CFD, Lorena Barba, Gilbert Forsyth - CFD Python

Table 3: Sources of PDE solver implementations used in experiments.

Comment corruption strategy. Comments are corrupted in order to create a conflict between the code logic and semantic meaning of comments. To corrupt a given snippet (the "receiver"), we select a "donor" snippet at random from valid, commented snippets from 1) a different PDE class and 2) different numerical method. The same donor is used for valid and invalid variants of the receiver. Next, donor comments are extracted in order and inserted exactly at the receiver's comment positions. If the donor has fewer comments than the receiver, comments are cycled, and extra donor comments are discarded. All dataset perturbations were performed using rule-based scripting rather than LLM automation to ensure reproducibility.

A.2 Models used in experimentation

Model Full Name on Huggingface	Family	Size	Type
`Qwen/Qwen2.5-Coder-7B-Instruct`	Qwen	7B	Coding
`Qwen/Qwen2.5-Coder-32B-Instruct`	Qwen	32B	Coding
`Qwen/Qwen3-32B`	Qwen	32B	Hybrid reasoning
`Qwen/QwQ-32B`	Qwen	32B	Dedicated reasoning
`deepseek-ai/DeepSeek-R1-Distill-Qwen-32B`	DeepSeek	32B	Dedicated reasoning
`meta-llama/Llama-3.1-8B-Instruct`	Meta	8B	General
`meta-llama/Llama-3.3-70B-Instruct`	Meta	70B	General
`mistralai/Mistral-Nemo-Instruct-2407`	Mistral	12B	General
`google/gemma-3-27b-it`	Google	27B	General
`microsoft/phi-4`	Microsoft	14B	General

Table 4: Models evaluated in free generation and multiple choice question experiments.

A.3 Full prompt templates

A.3.1 Free-generation experiment

```
You are analyzing a numerical simulation written in Python.
<code> {code} </code>
Answer the following about this simulation. Be concise.

Output only:
pde: ____
method: ____
behavior: ____
valid: ____

- pde: the type of PDE being solved
```

```
- method: numerical method(s) used - list all that apply
- behavior: dominant physical process(es) - list all that apply
- valid: Does this code run and produce a correct physical solution for the PDE?
```

A.3.2 Multiple choice experiment

PDE Class (4-way MC)

```
You are analyzing a numerical simulation written in Python.
<code> {code} </code>
What type of PDE is this simulation solving?

A) {opt_A} B) {opt_B} C) {opt_C} D) {opt_D}

Output only the letter of the correct answer.
```

True/False (process, method, validity)

```
You are analyzing a numerical simulation written in Python.
<code> {code} </code>
Is {candidate} one of the dominant physical processes in this simulation?

A) {opt_A} B) {opt_B}

Output only the letter of the correct answer.
```

Method and validity prompts are analogous. Options A/B are randomly assigned Yes/No per question instance. Options A/B/C/D are randomly assigned PDE class per question instance.

For each snippet, we ask 1 PDE class multiple choice question, 4 physical process True/False questions (regarding advection, diffusion, oscillation, and restoration), 3 numerical method True/False questions (regarding explicit, implicit and spectral), and 1 physically valid True/False question.

A.3.3 Belief revision experiment

```
Here is the runtime output from executing this simulation.
Fields below summarize execution of the script. Statistics are computed
on the variable identified as the primary solution array;
axis 0 of that array is treated as the time dimension.

<runtime_summary>
{traj_block}
</runtime_summary>

Re-evaluate your analysis using both the code and this runtime evidence.

Output only:
pde: ____
method: ____
behavior: ____
valid: ____
```

A.4 Aliases used for accuracy calculations

The following aliases are marked as correct in all accuracy calculations (for LLM free generation and belief revision).

Category	Canonical Label	Accepted Aliases
PDE Class	wave	wave, wave equation
	heat	heat, heat equation
	burgers	burgers, burgers equation
	navier-stokes	navier-stokes, navier stokes, ns equation, incompressible flow, navier stokes equations
Numerical Method	explicit	explicit, explicit euler, forward euler, rk4, runge-kutta, runge kutta
	implicit	implicit, implicit euler, crank-nicolson, crank nicolson, backward euler
	spectral	spectral, fft, fourier
Physical Behavior	oscillation	oscillation, oscillatory, vibration
	diffusion	diffusion, diffusive, spreading
	advection	advection, advective, transport, convection
	restoration	restoration, restoring, damping, decay

Table 5: Alias sets used for keyword-match scoring in the free-generation experiment. A response field is credited if any alias appears as a case-insensitive substring.

A.5 Ground-truth vs. corrupted variable name preference in PDE code

In this experiment, to understand how much context is needed for the LLM to learn the local variable name over the canonical variable in a PDE code snippet, we took the data subset consisting of valid corrupted code with and without ground-truth comments. This resulted in 32 PDE code snippets (4 each from Burgers, Heat, Navier-Stokes, and Wave). PDE state variables (u, v, w, p, T, phi, psi as well as their time-stepped variants such as u_n) and PDE parameter variables (such as physical and numerical constants including dx, dy, dt, nu, mu, alpha, flux) were identified by keyword and included as probe targets for this experiment. Loop indices, grid sizes, function names, and other non-PDE specific variables were not included in this experiment.

Every occurrence of every selected variable in the snippet was probed. For every probe, the context is all the code from the snippet up to, but not including, the variable token. We compute

$$\log P_{\text{gt}} = \sum_i \log P(\text{gt token}_i \mid \text{prefix})$$

and

$$\log P_{\text{corrupt}} = \sum_i \log P(\texttt{foobar\_N token}_i \mid \text{prefix}),$$

where log-probabilities are summed across all BPE tokens of the variable name, using BPE boundary detection.

We define

$$\log P_{\text{diff}} = \log P_{\text{gt}} - \log P_{\text{corrupt}}.$$

Thus, when $\log P_{\text{diff}}$ is negative, the model assigns higher probability to the corrupted variable name than to the ground-truth variable name. The code fraction indicates the character offset of the occurrence divided by the total number of characters in the snippet. We conduct this experiment on 3 Qwen models, including reasoning model `Qwen3-32B` and two coder models: `Qwen2.5-Coder-7B` and `Qwen2.5-Coder-32B`. Each snippet contributed a median of 51 probe points (IQR: 25–118, range: 15–410), varying with snippet length and variable density.

As shown in Figure 6, as the model accumulates more context, it better learns the local name of the variable and begins to predict it with increasing probability over the ground-truth canonical variable name. In Figure 7, we further disaggregate this analysis by PDE class, demonstrating some PDE classes, particularly Wave, show the smoothest growing preference for corrupted variable name over ground truth variable name.

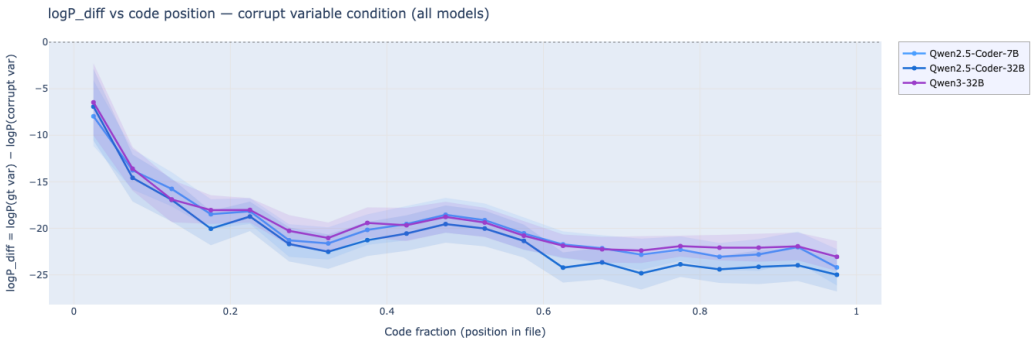

Figure 6: Log-probability preference for ground-truth variable names over corrupted names as a function of code position. Values below zero indicate that the model assigns higher probability to the corrupted variable name than to the ground-truth name. Shaded bands show 95% bootstrap confidence intervals. Across all models, the preference for corrupted names becomes stronger later in the snippet, indicating the model is learning the localized name for the variable.

Figure 7: Log-probability preference for ground-truth versus corrupted variable names across code position, separated by PDE class. Burgers and Wave show especially strong negative signals, indicating stronger preference for corrupted variable names as more code context accumulates.

A.6 Linear probe experiment details

Pooling strategy details. The first "layer" is the embedding of the tokenized code. The hidden states at each of the L layers were converted into two representations following two different pooling strategies. First, mean pooling takes the mean of all hidden states at code-token positions, and captures what information is encoded across code tokens. Last-token takes the hidden state of the final code token, and keeps track of what information accumulates at the end of the sequence.

Model dimensions. Both representations are D-length vectors, where D is the embedding size for a given model. We ran probes on `Qwen2.5-Coder-7B` ($L = 29$, $D = 3584$), `Qwen2.5-Coder-32B` and `QwQ-32B` ($L = 65$, $D = 5120$), focusing on results from `Qwen2.5-Coder-7B` in the main text.

Hyperparameters. We used the `scikit-learn` implementation of logistic regression, which natively handles both binarya and multinomial inputs. We used the `lbfgs` solver with maximum 500 iterations. Because the vectors are high-dimensional relative to input size ($N \ll D$), we investigated the effect of reducing D by projecting PCA to 20 dimensions and adjusting the C, the L2 regularization coefficient. Results for `Qwen2.5-Coder-7B` are shown in Figure 8. Changing hyperparameters did not have a significant effect on trends or layer ordering. PCA-20 degrades performance slightly, more so for `last_tok` and especially at low regularization. $C = 1$ on raw

vectors maximizes accuracy, so we used it for all experiments in Figure 4. For the multimodel pilot experiment in Figure 9, we projected representations for all models to PCA-20 for speed, using $C = 1$. The TF-IDF BoW baseline run with 500 features.

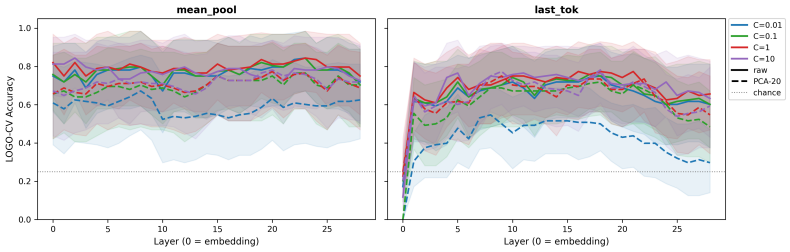

Figure 8: Effect of hyperparameters (regularization weight, original dimension (raw) vs. PCA-20, pooling method) on probe performance.

Multi-model comparisons. We ran a small pilot to compare representation similarity and probe performance across three models, shown in Figure 9. Figure 9 (a) shows that representations are shifted the most by variable corruption and the least by invalidity, and this likely represents lexical similarity. There is no significant difference in representation shift between `Qwen2.5-Coder-32B` (non-reasoning) and `QwQ-32B` (reasoning). Results in (a) are not comparable between 7B and 32B parameter models because representation shift is confounded by differences in D. Panels (b-d) (see Figure 9) show that larger scale models encode PDE class information at earlier layers compared to smaller scale models, and that there is again no significant difference in probe performance between `Qwen2.5-Coder-32B` (non-reasoning) and `QwQ-32B` (reasoning).

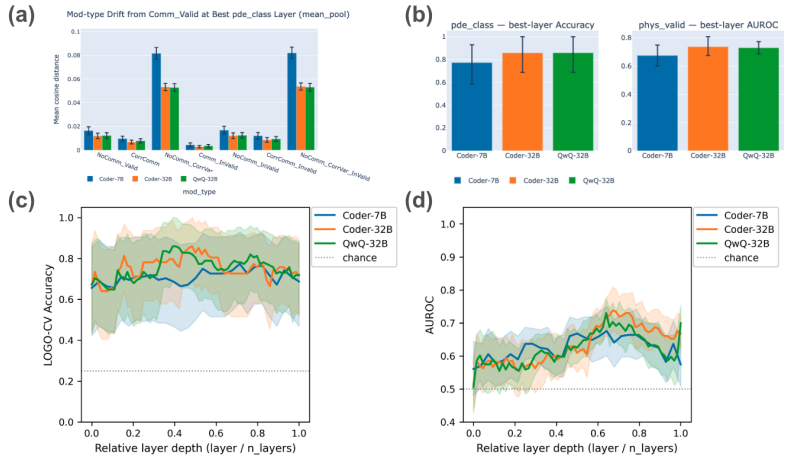

Figure 9: Results from representation similarity analysis and linear probes across 3 models.

A.7 Belief revision experiment details

A.7.1 Precomputing execution summaries

Rationale. PDE solver code typically does not return any outputs, instead simply updating all important arrays at each timestep. This produces a multi-dimensional trajectory. Instead of passing in full trajectories for all variables, which would stretch context limits, or asking models to design summary diagnostics, which opens the door to complex, non-reproducible agentic behavior, we precompute trajectory summaries based on a single "solution array." Most PDEs, including the ones in this dataset, have one or a few main variables of interest that dynamically evolve and reveal pathological behavior if present.

Solution array selection. The sandboxed environment enables clean dependency management including JAX on CPU, and subsequent extraction of solution arrays even if not returned. The

solution array for each code snippet is clear to a domain expert, but here is automatically selected by a heuristic ranking by 1) presence within time loop (`odeint`/`solve_ivp`) and 2) shape match with number of timesteps. Ties are broken by the size of the array, with one hard-coded fix for a Navier-Stokes snippet.

Structured fields. The structured summary reflects important flags and numerical considerations while maintaining neutral language. Execution summaries are not intended to be perfect, unambiguous ground-truth, but rather as runtime observations from a deterministic tool. The fields, their definitions, and any comments provided to the model are shown in the table below.

Field Name	Definition
`ran_to_completion`	whether script executed successfully
`main_array_name`	name of solution array in code
`shape`	shape of solution array
`has_nan`	NaN present in solution array
`has_inf`	Inf present in solution array
`finite_fraction`	fraction of non-Inf elements
`first_nonfinite_time_index`	first timestep where NaN/Inf appears ($\star$)
`max_abs_initial`	$\max \lvert a_0 \rvert$, max absolute value of array at first timestep
`max_abs_final`	$\max \lvert a_t \rvert$ at last timestep
`max_abs_over_time_sampled`	$\max \lvert a_t \rvert$ at 10 evenly spaced timesteps
`max_abs_before_nan`	max absolute value at last all-finite time step ($\star$)
`spike_ratio`	`max_abs_final / max_abs_initial` ($\star$)

Table 6: Trajectory summary fields. ($\star$) indicates definition provided to model.

A.7.2 Additional experimental results

Stratification of execution evidence. To further stratify the execution summaries for invalid code, we classified summaries as clear or ambiguous. Summaries of invalid code execution with clear anomalies such as `NaN` present, `Inf` present, `spike_ratio > 1000`, or `max_abs_final > 1e6` are labeled "clear." Other invalid summaries, which may contain other subtle numerical issues but lack obvious indicators, are labeled "ambiguous." While less obvious, these ambiguous issues are important as they cause impossible geometries in energy distributions or prevent evolution over time, which are physically impossible for any of the selected equations. Figure 10 shows that models always revise wrong answers when presented with clearly invalid trajectory evidence, but are much less swayed by ambiguous evidence.

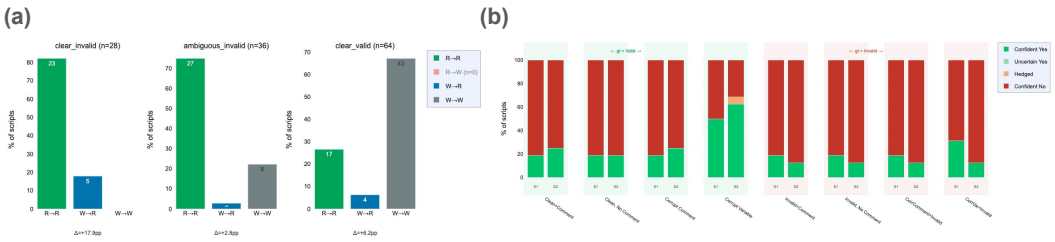

Figure 10: Further analysis from `Gemini-2.5-Flash` belief revision. In (a), we report percentages of belief-revisions transitions stratified by evidence ambiguity. In (b), we report model uncertainty at Stage 1 (code-only) vs. Stage 2 (code+execution summary), quantified by hedging, by perturbation type.

Mod Type	R→R (%)	R→W (%)	W→R (%)	W→W (%)	Δ Acc (pp)
Comm_Valid	19	0	6	75	+6.2
NoComm_Valid	19	0	0	81	+0.0
CorrComm	19	0	6	75	+6.2
NoComm_CorrVar	50	0	12	38	+12.5
Comm_InValid	81	0	6	12	+6.2
NoComm_InValid	81	0	6	12	+6.2
CorrComm_Invalid	81	0	6	12	+6.2
NoComm_CorrVar_InValid	69	0	19	12	+18.8
Overall	52	0	8	40	+7.8

Table 7: Full numbers for Gemini-2.5-Flash belief revision across perturbation types (shown previously in Figure 5). We report percentages of of belief-revision transitions (Right→Right, Wrong→Right, Right→Wrong, Wrong→Wrong) with accuracy change in the final column.